

Project - LTI identification

Emmeran COLOT

I. SYSTEM DESCRIPTION

The device studied throughout this project is a linear system with a static nonlinear feedback loop. It has a single input and a single output and the feedback is known to follow the input-output relation given by

$$y(t) = u^3(t). \quad (1)$$

The objective will be to identify the linear part of the system and to have a measure of the nonlinearity and noise levels. The knowledge of (1) will not be used, except for a small parenthesis about odd and even nonlinearities.

The measurement device (referred to as the *PXI setup*) allows to send only multisines and their rms value cannot exceed 1 V. Its ADCs have a range that can be chosen and can sample at high frequencies (> 1 GHz).

This report deliberately focusses on the theoretical points and the results rather than the implementation because the MatlabTM scripts and the measured data are available on GitHub¹.

II. EXCITATION SIGNAL

Before making any useful measurement, the parameters of the excitation signal must be chosen to match the DUT parameters. The rms of the applied signal must indeed be determined in order to use the full range of the ADC. This minimizes the quantization error and removes any saturation issue. The resonance frequency of the system must also be roughly known to determine the band of interest, making sure that the system's dynamics are measured.

A. Band of Interest

To determine the sampling frequency, a multisine excited up to 40 kHz has been applied to the DUT. Both input and output of the system are shown in the frequency domain in Fig. 1.

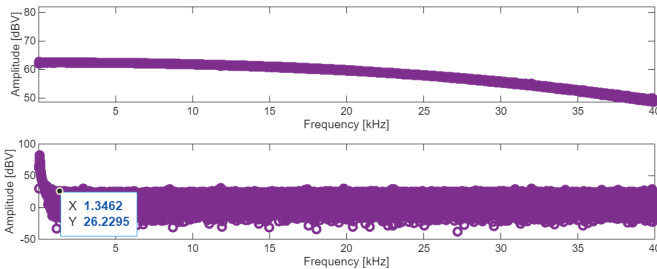


Fig. 1. Excitation spectrum (top) and output spectrum (bottom) for a fully excited multisine.

The measured input spectrum is not flat because of the ZOH: the true input is discretized by the *PXI setup* and the hold operation is equivalent to a sinc filtering in frequency. This effect could be reduced with a higher sampling frequency.

The output spectrum shows that the *interesting* part happens at frequencies below ~ 1.25 kHz. The excitation signals used will therefore stop at this frequency. To mitigate the effect of the ZOH, the sampling frequency is set to 5 kHz.

B. Amplitude

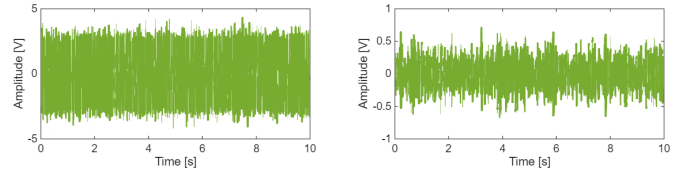


Fig. 2. Excitation signal (left) and output signal (right) in the time domain.

The same signal as the one measured for Sec. II-A has been used here, but shown in the time domain on Fig. 2. It can be seen that the input signal do not go higher than 5 V and the output voltage does not cross 1 V. This allowed to set the range of the ADCs of the *PXI setup*.

C. Transient Removal

Because only multisines can be applied, every measurement is done for multiple periods and the first ones are systematically removed until the transient is removed. To check this, the difference between each period is plotted, as shown in Fig. 3 and periods are removed until the shown plot is reduced to a flat curve.

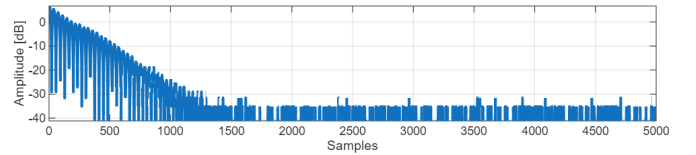


Fig. 3. Difference between 1st and 2nd periods of the output signal in time domain for a measure of the transient.

III. NONLINEAR DISTORTION AND NOISE ANALYSIS

Two methods are used to distinguish the nonlinear distortions from the noise. The *fast method* uses a single experiment and allows to split the output spectrum in linear, nonlinear and noise contributions. The *robust method* uses multiple experiments and computes an estimate of the variances of nonlinear distortion and noise together with an nonparametric estimate of the frequency response function (FRF).

¹<https://github.com/e-colot/LTI-identification>

A. Fast Method

The excitation signal is an odd multisine with holes, which is repeated a few times. This means that only odd frequencies are excited and some of those odd frequencies are also unexcited in order to have bins containing odd nonlinear distortion. During the excitation generation, 1 odd bin out of 4 is randomly chosen to be not excited.

The FFT of the multiple periods is taken, effectively dividing the spectral resolution as $\Delta_f = f_s/N$ with a larger N . The added bins are not linear combinations of input frequencies and therefore only contain noise, under the assumption that the DUT is a *PISPO*² system.

The spectrum of the output signal is then divided into 4 categories:

- **Linear**, for bins that were excited.
- **Even distortions**, for bins that are a combination of an even number of excited bins.
- **Odd distortions**, for bins that are a combination of an odd number of excited bins.
- **Noise**, for bins that are added by the multiple periods.

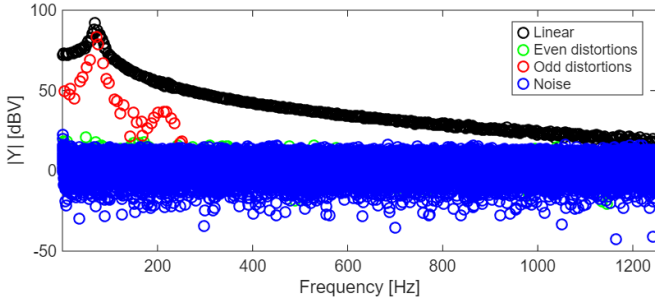


Fig. 4. Results of the fast method with the feedback given by (1). No FRF is computed because the one given by the robust method will be more useful in Sec. IV.

Fig. 4 shows that even distortion is close to the noise level and therefore does not impact much the signal. Odd distortion on the other hand is quite high and is close to the level of the linear part near its peak.

This behavior is explained by (1): because the third power of the output is fed back to the input, it contains sinusoids at the sum of 3 frequencies of the output. As the output only contains odd frequencies, it results in another odd frequency and the main nonlinearities therefore appears at odd bins.

To validate this explanation, the same experiment has been repeated with a modified system whose feedback follows

$$y(t) = u^2(t). \quad (2)$$

The results are shown in Fig. 5 and because feedback now excites even frequencies, the odd and even distortions are now at the same level.

B. Robust Method

The excitation is a full multisine (excited up to 1.25 kHz as explained in Sec. II-A) repeated P times. The phases are randomly selected and are recomputed for each of the M

²period in, same period out

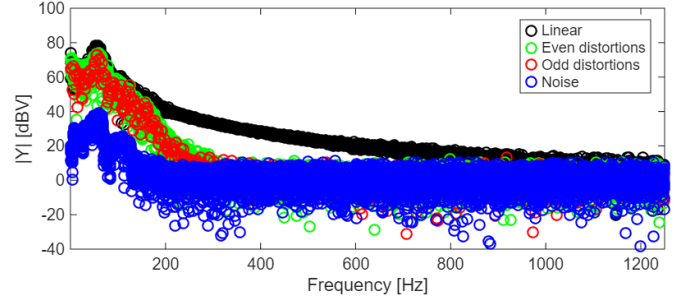


Fig. 5. Results of the fast method with the feedback given by (2).

realizations. The robust method relies on computing a FRF for each of the $P \times M$ periods, given by (3). The variance across the periods for realization m , computed by (5), is only due to noise. This is because the nonlinear contributions are deterministic and depend only on the input signal, which is the same for the whole realization.

$$\hat{G}^{[m,p]} = \frac{Y^{[m,p]}}{U^{[m]}} \quad (3)$$

$$\hat{G}^{[m]} = \frac{1}{P} \sum_{p=1}^P \hat{G}^{[m,p]} \quad (4)$$

$$\hat{\sigma}_n^2[m] = \frac{1}{P(P-1)} \sum_{p=1}^P \left| \hat{G}^{[m,p]} - \hat{G}^{[m]} \right|^2 \quad (5)$$

These noise variances on a single realization are then averaged across the realizations and divided by M because the averaging reduces the variance. This gives the total noise variance σ_n^2 :

$$\hat{\sigma}_n^2 = \frac{1}{M^2} \sum_{m=1}^M \hat{\sigma}_n^2[m]. \quad (6)$$

The averaged FRF of a single realization $\hat{G}^{[m]}$ does not contain noise (at least in expected value and for zero-mean noise). But between realizations, they have different nonlinear effects as the phases of the excitation signals have changed. The variance between those is therefore due to nonlinear distortion only and averaging it out gives a FRF estimate that is noiseless and purely linear, if M and P are large enough to be close to the expected value.

$$\hat{G}_{\text{BLA}} = \frac{1}{M} \sum_{m=1}^M \hat{G}^{[m]} \quad (7)$$

$$\hat{\sigma}_{\text{dist}}^2 = \frac{1}{M(M-1)} \sum_{m=1}^M \left| \hat{G}^{[m]} - \hat{G}_{\text{BLA}} \right|^2 \quad (8)$$

The estimated FRF and the distortion estimates are shown on Fig. 6 with the total variance $\hat{\sigma}_{\text{tot}}^2$ being the sum of both.

IV. BEST LINEAR APPROXIMATION

A linearized model of the system can be determined to approximate the DUT as an LTI. The best linear approximation, or BLA, is therefore computed as the "best fit". Because nonlinear distortions are dependent on the input signal, the

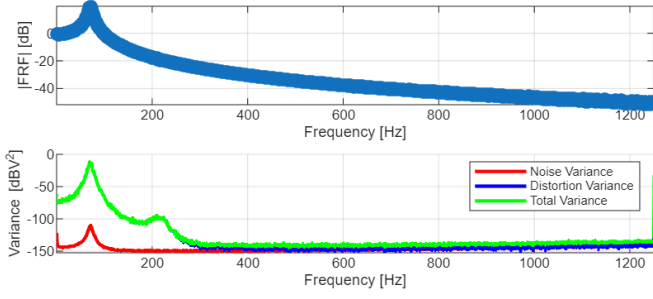


Fig. 6. Results of the robust method, with the FRF estimate (top) and the estimated variances (bottom). The total variance is simply equal to the sum of the noise and distortion variances

BLA computed here is valid for excitation signals with a similar *Riemann equivalent spectrum*.

It is here mentioned but as there is no use of the determined model, there is no interest in defining the category of excitations for which the model can be considered valid.

A. Nonparametric Model

The result of the robust method shown in Fig. 6 is, in fact, an estimated BLA and it already comes with its uncertainty. Because of the successive averaging, it should be able to remove noise and nonlinear distortions which are zero-mean³.

B. Generalities About Parametric Identification

Based on the nonparametric model of Sec. IV-A, a parametric model can be derived by making a choice for the model structure. A practical choice is a polynomial model (9), in which n_a and n_b are the model order.

$$G(s) = \frac{\sum_{k=0}^{n_b} b_k s^k}{\sum_{k=0}^{n_a} a_k s^k}. \quad (9)$$

Different estimators are used but first, a few tricks are used for each of them and those are explained here.

a) *Normalization by rms scaling*: Each estimator has a regressor matrix (H or J), whose columns often have different orders of magnitude. This results in a poor numerical conditioning. To solve this, a change of coordinates is applied with the scaling matrix S , which scales the columns. The original coordinates H (or J) and θ are replaced by $\tilde{H}$ (or $\tilde{J}$) and $\tilde{\theta}$.

$$y = H\theta \quad (10)$$

then becomes

$$y = \tilde{H}\tilde{\theta} \quad \text{with} \quad \tilde{H} = HS \quad (11)$$

$$\Leftrightarrow \tilde{\theta} = S^{-1}\theta \quad (12)$$

$$\Leftrightarrow \theta = S\tilde{\theta} \quad (13)$$

S has been chosen to be a diagonal matrix with as elements the inverse of the rms of the columns of H , such that it is invertible

$$S = \text{diag}\left(\frac{1}{\text{rms}(H_{:,1})}, \dots, \frac{1}{\text{rms}(H_{:,n})}\right). \quad (14)$$

³It is only true in expected value so it can only be approached using a discrete number of periods and realizations.

b) *Use of the nonparametric estimate*: All the estimators are relying on approaching (15), which arises from the definition of a transfer function. The key idea is to notice that because

$$Y(k) = G_{\text{BLA}}(k)U(k), \quad (15)$$

a clever choice of U simplifies the choice of Y , as

$$U(k) = 1 \quad \forall k \Rightarrow Y(k) = G_{\text{BLA}}(k). \quad (16)$$

c) *Forcing real valued parameters*: A real system is described by real coefficients for a polynomial model (9) so θ must be real. The regressor must therefore be changed, meaning y and H are replaced by a concatenation of their real and imaginary parts called y_{re} and H_{re} as follows

$$y_{\text{re}} = \begin{bmatrix} \Re\{y\} \\ \Im\{y\} \end{bmatrix} \quad \text{and} \quad H_{\text{re}} = \begin{bmatrix} \Re\{H\} \\ \Im\{H\} \end{bmatrix}. \quad (17)$$

The covariance matrix C_J used in the GTLS estimator in Sec. IV-C3 must also be replaced with

$$C_{J_{\text{re}}} = \Re\{C_J\}. \quad (18)$$

C. Parametric Estimators

1) *Linear Least Squares Estimator (LLS)*: It minimizes

$$V_{\text{LLS}} = \sum_k |E(k)|^2, \quad (19)$$

where $E(k) = A_k Y(k) - B_k U(k)$. The optimal parameters θ_{LLS} is found by writing the problem as

$$Y = H\theta_{\text{LLS}}, \quad (20)$$

under the assumption that a_0 is different from 0. With all the previous tricks, it becomes

$$G_{\text{BLA}_{\text{re}}} = \tilde{H}_{\text{re}} \tilde{\theta}_{\text{LLS}} \quad (21)$$

and its solution is given by

$$\theta_{\text{LLS}} = S \left(\tilde{H}_{\text{re}} \times S \right)^\dagger G_{\text{BLA}_{\text{re}}}. \quad (22)$$

2) *Total Least Squares Estimator (TLS)*: It minimizes the same cost function

$$V_{\text{TLS}} = \sum_k |E(k)|^2, \quad (23)$$

but there is another constraint which does not require a_0 to be different from 0, stated as

$$\|\theta_{\text{TLS}}\|_2^2 = 1. \quad (24)$$

The estimated parameters are then

$$\theta_{\text{TLS}} = S V_{\tilde{J}_{\text{re}}[:, \text{end}]}, \quad (25)$$

where J is the regression matrix such that

$$E = J\theta_{\text{TLS}} \quad (26)$$

and $V_{\tilde{J}_{\text{re}}}$ is the right matrix of the singular value decomposition (svd) of $\tilde{J}_{\text{re}}$. This estimator is not consistent because the covariance of ΔJ is different from the identity matrix, where ΔJ contains the noisy contributions of J .

3) *Generalized Total Least Squares Estimator (GTLS)*: It allows to overcome the inconsistency of the TLS estimator by using square of the covariance matrix of ΔJ , namely

$$C_J^{1/2} = \sqrt{\mathbb{E}\{\Delta J^H \Delta J\}}, \quad (27)$$

as weight for J . The cost function to minimize is

$$V_{\text{GTLS}} = \|J\theta_{\text{GTLS}}\|_2^2, \quad (28)$$

under the constraint that

$$\|C_J^{1/2}\theta_{\text{GTLS}}\|_2^2 = 1. \quad (29)$$

A small parenthesis about the construction of C_J is here made. Because ΔJ is only due to noise, it can be expressed as

$$\Delta J = \begin{bmatrix} \vdots & & \vdots \\ N_{\text{GBLA}}(k) & \cdots & s_k^{n_a} N_{\text{GBLA}}(k) & 0 \\ \vdots & & \vdots \end{bmatrix}. \quad (30)$$

Its covariance is therefore

$$C_J(i, j) = \begin{cases} \sum_k s_k^{i+j-2} \hat{\sigma}_{\text{tot}}^2(k) & \text{if } i, j \leq n_a + 1 \\ 0 & \text{otherwise} \end{cases}. \quad (31)$$

With X_{re} the right matrix of the generalized svd (gsvd)⁴ of $\tilde{J}_{\text{re}}$ and $C_{\tilde{J}_{\text{re}}}^{1/2}$,

$$\theta_{\text{GTLS}} = S(X_{\tilde{J}_{\text{re}}}^T)^{-1} \frac{1}{S_{\tilde{J}_{\text{re}}}(i, i)}. \quad (32)$$

To determine i , the middle matrices $C_{\tilde{J}_{\text{re}}}$ and $S_{\tilde{J}_{\text{re}}}$ of the gsvd are used:

$$i = \arg \min_i \frac{C_{\tilde{J}_{\text{re}}}(i, i)}{S_{\tilde{J}_{\text{re}}}(i, i)} \quad (33)$$

4) *Bootstrapped Total Least Squares Estimator (BTLS)*: All the previous estimators tend to overemphasize high frequency errors as the regressors become large at higher frequencies. The BTLS estimator weights it with the inverse of the standard deviation of the error, to a power r chosen in $[0; 1]$.

In practice, it is the same as a GTLS estimator where

$$\tilde{J}_{\text{re}} \mapsto W \tilde{J}_{\text{re}} \quad (34)$$

with

$$W = \text{diag}(\sigma_e^{-2r}). \quad (35)$$

Equation (32) is then applied iteratively until the new cost is greater or equal to the previous one. It must be specified that the variance of the error is computed based on the previous parameters.

⁴ $J = U_J C_J X_J^T$, $C_J^{1/2} = V_J S_J X_J^T$

5) *Maximum Likelihood Estimator (ML)*: This estimator maximizes the probability that the parameters θ_{ML} generated the measured data, and is therefore mathematically the best. It is however nonlinear and must be solved iteratively, just as for the BTLS. Its cost function is

$$V_{\text{ML}} = \frac{1}{2} \sum_k \frac{|E(k)|^2}{\sigma_e^2(k)} \quad (36)$$

and the parameter update $\Delta\theta$ with the *Newton-Gauss* algorithm is given by

$$\Delta\theta = -J_{\text{re}}^{\dagger} \epsilon_{\text{re}}, \quad (37)$$

where J is the jacobian of ϵ , which is itself defined as

$$\epsilon = \frac{e}{\sigma_e}. \quad (38)$$

No scaling is applied because it does not help the conditioning. It has been tried with rms and 2-norm normalization but none of them helped. A deeper analysis revealed that the blocks of zeros in (42) leads to rank deficiencies of the jacobian, which triggers warnings during Matlab computations.

The main difficulty in the implementation of the ML estimator is in the computation of the jacobian J . It needs A_k and B_k such that

$$\begin{aligned} E(k) &= A_k Y(k) - B_k U(k) \\ &= A_k G_{\text{BLA}}(k) - B_k. \end{aligned} \quad (39)$$

From the model (9), these are given by

$$A_k = \sum_i a_i (s_k)^i \quad \text{and} \quad B_k = \sum_i b_i (s_k)^i. \quad (40)$$

The gradient of A_k and B_k with regards to θ is also needed and there is therefore a need to define θ as

$$\theta = [a_0 \quad \cdots \quad a_{n_a} \quad b_0 \quad \cdots \quad b_{n_b}]^T. \quad (41)$$

In the following, it will often be written as $\theta = [\theta_a \quad \theta_b]^T$ to ease the notation but θ_a and θ_b still contains multiple elements.

The derivatives with respect to θ is computed by deriving it with respect to θ_a and θ_b separately but they are concatenated back afterwards.

$$\begin{aligned} \frac{\partial A_k}{\partial \theta_a} &= [1 \quad s_k \quad \cdots \quad s_k^{n_a}]^T, \quad \frac{\partial A_k}{\partial \theta_b} = 0_{(n_b+1) \times 1} \\ \frac{\partial B_k}{\partial \theta_a} &= 0_{(n_a+1) \times 1}, \quad \frac{\partial B_k}{\partial \theta_b} = [1 \quad s_k \quad \cdots \quad s_k^{n_b}]^T \end{aligned} \quad (42)$$

The rest of the computation of the jacobian simply follows what has been explained in the lectures so it is not detailed in this report.

The norm of the parameter change per iteration $\Delta\theta$ for is shown on Fig. 7. Its low value gives the intuition that the result of Newton-Gauss is already close to the optimal parameters. The Levenberg-Marquardt algorithm is then applied but will be discussed in Sec. VII-D, following the chronological order of the implementation and the reasons that pushed towards its implementation.

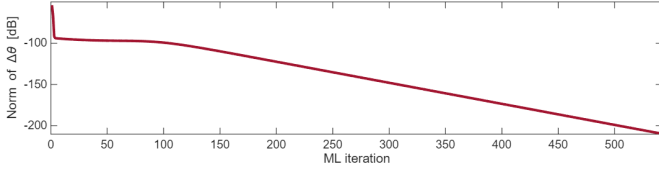


Fig. 7. Norm of the parameter change in function of the iteration for the optimal order of the ML algorithm.

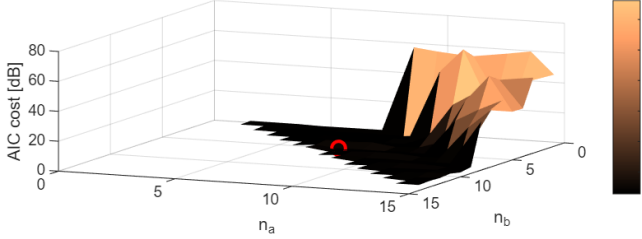


Fig. 8. AIC cost depending on the order of the transfer function for the TLS estimator. The optimal order is circled in red. n_a and n_b are going up to 15.

V. MODEL ORDER SELECTION

The estimators described in Sec. IV-B require a model order (n_a, n_b) to determine the right parameters $\theta = [a_0, \dots, b_{n_b}]$ of (9).

The AIC criterion is therefore used. It depends on the cost of the estimator V_{est} and the order of the model, giving an *AIC cost*, given by

$$V_{\text{AIC}} = V_{\text{est}} \left(1 + \frac{n_a + n_b}{N_e} \right). \quad (43)$$

This is computed for various values of n_a and n_b and the optimal order is the one with the smallest *AIC cost*.

The values of the order are restricted to transfer functions with more poles than zeros ($\Rightarrow n_a \geq n_b$) to have a stable system.

VI. RESULTS

The AIC criterion described in Sec. V has been applied to every estimator of Sec. IV-B and it gives for each of them a 3D plot like the one shown in Fig. 8.

The resulting FRFs are plotted in Fig. 9. It can be seen that the LLS and TLS estimators have *bumps* where poles and zeros nearly cancel each other's. The three last estimators (GTLS, BTLS and ML) superpose perfectly, making them undistinguishable on the graph.

Fig. 10 shows the optimal order that minimizes the AIC cost (43). A few observations can be made:

- The LLS and TLS estimators both lay close to the limit $n_a = n_b$. Fig. 11 confirms the hypothesis of imperfect pole-zero cancellations emitted based on the bumps of the FRF.
- GTLS and BTLS have a more intermediate number of parameters with fewer zeros.
- The ML estimator uses a low number of poles and zeros, yet achieve a performance similar to the GTLS on Fig. 9, which was expected as it is mathematically the best.

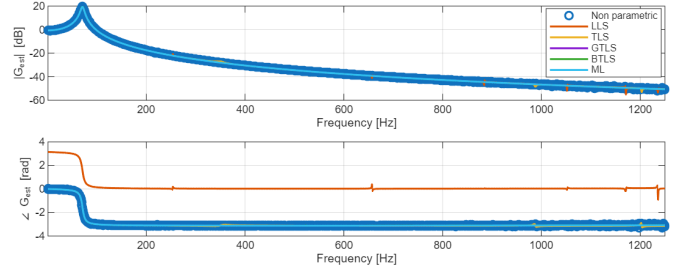


Fig. 9. Superposition of the estimated parametric FRFs amplitude (top) and phase (bottom), together with the nonparametric estimate.

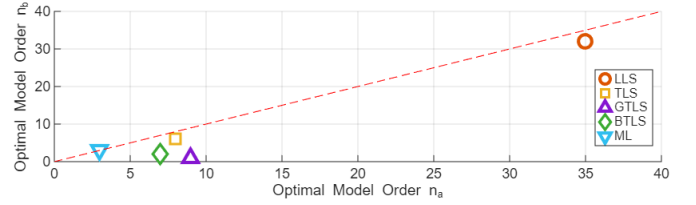


Fig. 10. Optimal orders for each estimator, according to the AIC criterion. The red dashed line is the stability limit $n_a = n_b$. n_a and n_b are going up to 15.

From Fig. 11, it can be seen that some poles are unstable (positioned in the right half-plane) and it is the case for every estimator. This will be resolved in Sec.VII.

VII. DEEPER ANALYSIS OF THE POLES

A. Poles position uncertainty

The covariance of the poles (and zeros) of the ML estimator can be approximated by

$$\text{cov}(p) \simeq \frac{dp}{d\theta_{\text{ML}}} \left(2J_{\text{re}}^H J_{\text{re}} \right)^{-1} \frac{dp^H}{d\theta_{\text{ML}}}, \quad (44)$$

where $\frac{dp}{d\theta_{\text{ML}}}$ is based on how the parameters $[a_0, \dots, a_{n_a}]$ relate to the poles $[p_0, \dots, p_{n_a}]$.

The resulting standard deviation on the unstable pole (the one in the right half-plane) is large as shown in Table I. The pole is however far from the origin, which means it is unlikely to move to the left half-plane.

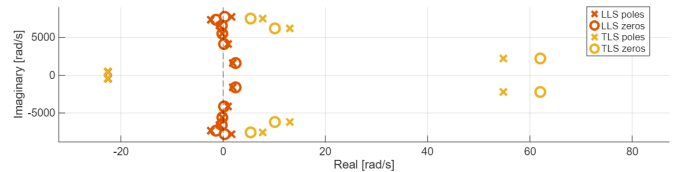


Fig. 11. Poles and zeros of the LLS and TLS estimators. Only the ones close to the origin are shown to visualize the close to perfect pole-zero cancellation.

TABLE I
STANDARD DEVIATION OF POLES ESTIMATED BY ML, IN RAD/S

Pole location	Std. deviation
49 334.50	14 578.58
$-22.23 + j 445.33$	$0.18 + j 0.18$
$-22.23 - j 445.33$	$0.18 + j 0.18$

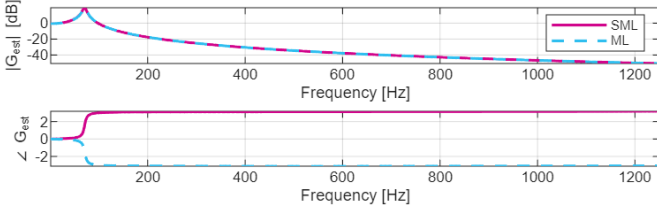


Fig. 12. Comparison of the estimated FRF of the ML and the SML algorithms.

B. Stable ML Estimator (SML)

To obtain a parametric FRF estimate where the poles are stable, the model (9) is changed so the relation between parameters and poles becomes explicit:

$$G(s) = \frac{\sum_{k=0}^{n_b} b_k s^k}{\prod_{k=1}^{n_a} (s - \alpha_k)}, \quad \text{with } \theta = [\theta_\alpha \quad \theta_b]^T \quad (45)$$

The constraint on the real part of the poles can be enforced after each parameter update but before looking to the way of constraining them, let's see how the use of the new model (45) impacts the ML algorithm described in Sec. IV-C5.

B_k and it's derivatives stay unchanged, except that the length of θ now becomes $n_a + n_b + 1$, because explicitly writing the poles removes a parameter⁵. On the other hand, we now have

$$A_k = \prod_i (s_k - \alpha_i), \quad (46)$$

and it's gradient along θ_α :

$$\frac{\partial A_k}{\partial \theta_\alpha} = -A_k \left[\frac{1}{(s_k - \alpha_1)} \quad \cdots \quad \frac{1}{(s_k - \alpha_{n_a})} \right]^T. \quad (47)$$

Once the parameters update is computed by (37), $\Delta\theta$ is scaled such that the poles stay in the left half-plane. It is implemented as follows:

$$\theta^{i+1} = \theta^i + k\Delta\theta \quad (48)$$

where

$$k = \min \left(1, \frac{-\Re\{\alpha_j^i\}}{\Re\{\alpha_j^{i+1}\} - \Re\{\alpha_j^i\}} \right) \quad (49)$$

for every pole α_j such that $\Re\{\alpha_j^{i+1}\} > 0$. A second approach could have been to limit only the poles that would cross the imaginary axis but this would have been equivalent to not follow the gradient anymore, losing the interest of using a maximum likelihood estimator. The resulting FRF is compared with the one obtained without constraining the poles in Fig. 12.

The SML estimator is however not useful. If it does impose poles with a negative real value, it cannot be applied while enforcing real parameters as the poles $\alpha_1, \dots, \alpha_{n_a}$ need to be complex. The resulting parameter vector can therefore not be written again in the general form (9) with real values for a_k and b_k .

⁵It is worth noticing that this allows to remove the constraint on $\|\theta\|_2^2$.

TABLE II
COST COMPARISON OF THE ML ESTIMATORS

Estimator	ML Cost
ML	822.34
SML	824.52
SRML	827.36
OML	827.36

C. Stable Real ML Estimator (SRML)

This section starts from what has been done in Sec. VII-B and improves it to end up with real parameters, while maintaining stable poles, hence the name *Stable Real ML* estimator.

The transfer function takes the form

$$G(s) = \frac{\sum_{k=0}^{n_b} b_k s^k}{\prod_{k=1}^{n_{a, \text{re}}} (s - a_k) \prod_{k=1}^{n_{a, \text{cplx}}} (s^2 - 2s\alpha_k \cos \phi_k + \alpha_k^2)}, \quad (50)$$

such that it's poles are given by a_k and $\alpha_k e^{\pm j\phi_k}$. The parameters have now become

$$\theta = [a_k \quad \alpha_k \quad \phi_k \quad b_k]^T. \quad (51)$$

This new model implies the following changes:

$$A_k = \prod_i (s_k - a_i) \prod_i (s_k^2 - 2s_k \alpha_i \cos \phi_i + \alpha_i^2), \quad (52)$$

$$\frac{\partial A_k}{\partial a_i} = -A_k \left[\cdots \quad \frac{1}{(s_k - a_i)} \quad \cdots \right]^T, \quad (53)$$

$$\frac{\partial A_k}{\partial \theta_\alpha} = 2A_k \left[\cdots \quad \frac{\alpha_i - s_k \cos \phi_i}{(s_k^2 - 2s_k \alpha_i \cos \phi_i + \alpha_i^2)} \quad \cdots \right]^T, \quad (54)$$

$$\frac{\partial A_k}{\partial \theta_\phi} = 2A_k \left[\cdots \quad \frac{s_k \alpha_i \sin \phi_i}{(s_k^2 - 2s_k \alpha_i \cos \phi_i + \alpha_i^2)} \quad \cdots \right]^T. \quad (55)$$

This model makes sure the complex poles are complex conjugates, while keeping the parameters real. However, constraining the poles to the left half-plane is still done by limiting $\Delta\theta$ with (49), which is no longer following the gradient. The amplitude and phase parametrization makes the interpolation harder: only ϕ_k should be limited, except if the amplitude changes sign. This could make the estimator differ from the optimal stable solution, which needs to be solved.

Before adding a last layer of complexity to cure this, let's rapidly discuss the results of the SML and SRML estimators. First visually, the SRML estimate shown on Fig. 13 has a worse phase match than Fig. 12, while both seem to follow well the amplitude. It can be precised that the ML model is used as reference instead of the measured points as it is known that the ML estimate is mathematically optimal.

More rigorously, the final ML cost of all the maximum likelihood estimators, computed with (36), are shown in Table II and it can be seen that the SRML performs a bit worse than the SML. This is expected as the SRML algorithm finds a minimal cost in a reduced solution set, which does not contain either the unstable solution of ML or the unreal solution of SML.

D. Optimal ML Estimator (OML)

This estimator is called *optimal ML* as it removes the need for hand crafted constraints. This is done by manipulating (50)

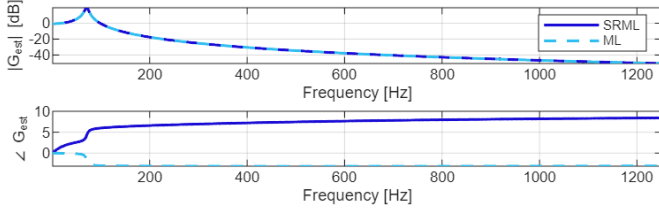


Fig. 13. Comparison of the estimated FRF of the ML and the SRML estimators.

so its poles can no longer have a positive real part. The resulting transfer function is:

$$G(s) = \frac{\sum_{k=0}^{n_b} b_k s^k}{\prod_{k=1}^{n_{a, \text{re}}} (s + a_k^2) \prod_{k=1}^{n_{a, \text{cplx}}} (s^2 + 2s\alpha_k^2 + \alpha_k^4 + \phi_k^2)}. \quad (56)$$

The resulting poles are in $-a_k^2$ and $-\alpha_k^2 \pm j\phi_k$. For the same parameter vector θ as in (51),

$$A_k = \prod_i (s_k + a_i^2) \prod_i (s_k^2 + 2s_k\alpha_i^2 + \alpha_i^4 + \phi_i^2), \quad (57)$$

$$\frac{\partial A_k}{\partial \theta_a} = 2A_k \left[\cdots \frac{a_i}{(s_k + a_i^2)} \cdots \right]^T, \quad (58)$$

$$\frac{\partial A_k}{\partial \theta_\alpha} = 2A_k \left[\cdots \frac{s_k\alpha_i + 2\alpha_i^3}{(s_k^2 + 2s_k\alpha_i^2 + \alpha_i^4 + \phi_i^2)} \cdots \right]^T, \quad (59)$$

$$\frac{\partial A_k}{\partial \theta_\phi} = 2A_k \left[\cdots \frac{\phi_i}{(s_k^2 + 2s_k\alpha_i^2 + \alpha_i^4 + \phi_i^2)} \cdots \right]^T. \quad (60)$$

It uses as initial parameters the results of the SRML estimator of Sec. VII-C. Even though it was never mentioned in this report, the norm of $\Delta\theta$ at the end of the Newton-Gauss iteration has always been checked and was determined to be low enough so a second loop using the Levenberg-Marquardt (LM) algorithm was never used.

This norm was however not negligible with the OML estimator so the LM was implemented and "retro-fitted" to each ML estimator. The most complicated part was the determination of a convergence condition and it was decided to be a minimal threshold on the relative change of the parameters:

$$\frac{\|\Delta\theta\|}{\|\theta\|} \leq 10^{-12} \quad (61)$$

There is no cost improve between this estimator and the SRML, as shown in Table II and they are in fact finding the same exact solution. This is once again explained by thinking about the available solution set, which is exactly the same for both.

This does not imply those are the same and they could differ slightly when poles approach the imaginary axis. As discussed in Sec. VII-C, the interpolation in case of constraints violation is badly handled and could therefore be suboptimal.

E. Conclusion on the Different ML Estimators

The optimal order of the ML estimators shown on Fig. 15 shows that the SRML and OML estimators have a larger order than the ML estimate. It is not a surprise and it even proves that the *classical way* of stabilizing a system by symmetrizing unstable poles along the imaginary axis is not the optimal way,

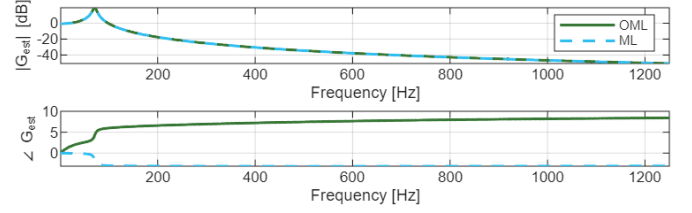


Fig. 14. Comparison of the estimated FRF of the ML and the optimal ML algorithms.

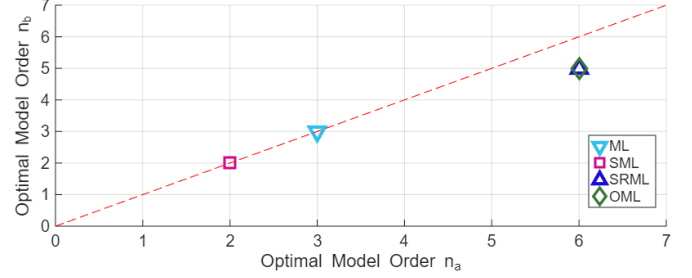


Fig. 15. Optimal orders for each variant of the ML estimator, according to the AIC criterion. The red dashed line is the stability limit $n_a = n_b$.

in which case the model order would have been the same for all the estimators.

A final note can be made about the uncertainty on the poles of the SRML and OML estimators. Their transfer function models (50) and (56) link directly the parameters to the poles so their gradients $\partial p / \partial \theta$ are block matrices. They are structurally described here, just to give an insight on how they are built.

$$\frac{\partial p_{\text{SRML}}}{\partial [\theta_a \ \theta_\alpha \ \theta_\phi]} = \begin{bmatrix} I & \text{diag}_i(e^{j\phi_i}) & \text{diag}_i(j\alpha_i e^{j\phi_i}) \\ \text{diag}_i(e^{-j\phi_i}) & \text{diag}_i(-j\alpha_i e^{-j\phi_i}) \end{bmatrix}$$

$$\frac{\partial p_{\text{OML}}}{\partial [\theta_a \ \theta_\alpha \ \theta_\phi]} = \begin{bmatrix} \text{diag}_i(-2a_i) & \text{diag}_i(-2\alpha_i) & jI \\ \text{diag}_i(-2\alpha_i) & -jI \end{bmatrix}$$

And finally, the resulting standard deviation of the poles are given in Table III. It can be observed that there is a factor 2 between the standard deviation of the SRML and the one of OML, and this only for the real part of their complex poles.

It is simply due to the squaring of α_k in (56), which makes the link between pole position and parameter behave differently.

TABLE III
STANDARD DEVIATION OF POLES ESTIMATED BY SRML AND OML, IN RAD/S

Pole location	Std. deviation-SRML	Std. deviation-OML
-4250.10	275.23	275.23
-140.62	27.52	27.52
$0 + j 5770.62$	$1.68 + j 1.77$	$3.35 + j 1.77$
$-22.17 + j 445.12$	$0.19 + j 0.21$	$0.37 + j 0.21$
$0 - j 5770.62$	$1.68 + j 1.77$	$3.35 + j 1.77$
$-22.17 - j 445.12$	$0.19 + j 0.21$	$0.37 + j 0.21$